INFO410 – Structure de données classiques

Examen

1. Les arbres (et plus si affinités) [6 points]

On considère la structure suivante :

```
type noeud;
type arbre is access noeud;
type noeud is record
   val: integer;
   fg, fd: arbre;
end record;
```

```
type elem;
type liste_arb is access elem;
type elem is record
   n: integer;
   a: arbre;
   suiv: liste_arb;
end record;
```

Faire un schéma représentant un exemple d'objet de type `liste_arb` avec 3 éléments différents.

Ecrire de manière récursive une fonction qui calcule la somme de toutes les valeurs de l'arbre qui sont supérieures ou égales à un X donné.

```
function somme_sup (A: in arbre; X: in integer) return integer is …
```

On complète la structure précédente avec celle-ci :

```
type element;
type liste is access element;
```

```
type element is record
   info: integer;
   suiv: liste;
end record;
```

En vous servant de la fonction `somme_sup`, écrire de manière récursive une fonction `mega_somme` qui crée une liste d'entiers de même longueur que `L`. Chaque entier est le résultat de la fonction `somme_sup` appliquée à l'élément correspondant de la liste. Ainsi, le 1er entier sera égal à la somme de tous les entiers du 1er arbre supérieurs ou égaux au nombre n du 1er nœud…

```
function mega_somme (L: in liste_arb) return liste is …
```

2. Spécification et corps de package [6 points]

On donne le type suivant :

```
type vecteur is array(1..N) of float;
```

Ecrire la spécification et le corps (bien sûr) du package `liste_circu_vecteur` qui permet de traiter des listes circulaires de « vecteurs » et permet de :

- Fournir un type `liste_vect` qui représente une liste circulaire de vecteurs sans que l'on sache comment il est constitué ;
- Fournir une constante `liste_vect_vide` ;
- Fournir une fonction qui compte le nombre de vecteurs dont le deuxième coefficient est positif ;
- Fournir une procédure qui insère un vecteur en tête de liste.

A chaque fois que vous traitez un vecteur, il sera rempli. Vous n'avez pas à vous en occuper.

```
package liste_circu_vecteur is
   N: constant integer := 20;
   type vecteur is array(1..N) of float;
…
private
…
end liste_circu_vecteur;
```

Aucune procédure ou fonction récursive dans cette partie.

```
package body liste_circu_vecteur is
…
end liste_circu_vecteur;
```

3. Itératif ET récursif [4 points]

On considère les déclarations suivantes :

```
type noeud;
type liste is access noeud;
type noeud is record
   x, y: integer;
   b: boolean;
   suiv: liste;
end record;
```

Ecrire la fonction bizarre qui fait la somme de toutes les valeurs quand x = y moins le nombre de nœuds où b est vrai d'une liste pointée par L, de manière **itérative puis récursive**.

Par exemple, si la liste contient (3, 3, false), (5, 4, true), (6, 6, true), (8, 7, false), (4, 5, true), (5, 7, false), (-3, -3, false) et (4, 0, true), la fonction rend (3 + 6 – 3) – 4 = 2.

4. Un tableau de piles [4 points]

On se propose d'appliquer un traitement particulier à une suite de termes entrée par l'utilisateur et terminée par « *. ». Ces traitements sont appliqués lorsque l'on rencontre un opérateur spécial qui commence par « * » et suivi d'une lettre (en minuscule) ou un point.

« *a » affiche tous les termes rencontrés jusqu'à maintenant et qui commencent par un « a » dans l'ordre inverse puis les supprime ;
« *b » affiche les termes qui commencent par un « b » dans l'ordre inverse et les supprime ;
« *c » idem … jusqu'à « *z »
« *. » termine la phrase et affiche tous les termes rencontrés non traités commençant par « a » dans l'ordre inverse, puis ceux commençant par « b » dans l'ordre inverse, « c » … jusqu'à « z ».

Tous les termes qui commencent par autre chose qu'une minuscule sont ignorés.

Exemple à lire attentivement pour voir si vous avez bien compris l'énoncé :
Si l'utilisateur entre : *Il est facile De Penser Que *t cet examen Dure vraiment *v 1h30 *.*
Le programme affiche : *vraimentcetexamenestfacile*

On remarque que :

- *Il*, *De*, *Penser*, *Que*, *Dure* et *1h30* ont été ignorés
- « *t » n'a rien déclenché puisqu'aucun mot commençant par un « t » n'a été rencontré
- « *v » a déclenché l'affichage de *vraiment*
- « *. » a déclenché l'affichage des termes restants (tous les termes sauf *vraiment*)

/!\ Il est interdit d'utiliser une chaîne ou un tableau (le non-respect de cette consigne entraine 0).

Ecrire la procédure principale qui réécrit une suite de termes selon la méthode décrite ci-dessus. Vous avez à votre disposition une fonction et un package qui vous sont conseillés d'utiliser.

La fonction `get_terme`, qui ne doit pas vous paraitre inconnue, sait sauter les espaces et retourne un terme à chaque appel. En reprenant l'exemple, au 1er appel `"Il"`, puis `"est"`, ensuite `"facile"`…

```
function get_terme return unbounded_string;
```

Le package « piles » permet de disposer de :

- un type `pile` privé (dont chaque élément est une chaine de caractères)
- une fonction `init_pile: pile` qui crée une pile vide
- une fonction `est_vide(P: in pile): boolean` qui rend vrai si la pile est vide, faux sinon
- une procédure `empile(P: in out pile; X: in unbounded_string)` qui empile X au sommet de la pile
- une procédure `depile(P: in out pile; X: out unbounded_string)` qui dépile le sommet et le range dans X

N'oubliez pas de préciser les packages que vous allez utiliser, de déclarer le type de tableau nécessaire, d'initialiser vos structures de données et de commenter si besoin. Il est également recommandé d'écrire un sous-programme afin d'éviter de dupliquer du code qui se ressemblerait… Enfin, vous ne serez pas pénalisé sur les conversions entre les types `string` et `unbounded_string`.